

# PyUltraLight2 (BHPaper) — User Manual

Auckland Cosmology Group

Version 3.0.0 “First Light” — September 24, 2026



## Contents

|          |                                      |          |
|----------|--------------------------------------|----------|
| <b>1</b> | <b>Overview</b>                      | <b>1</b> |
| <b>2</b> | <b>Installation and Dependencies</b> | <b>2</b> |
| <b>3</b> | <b>Quick Start</b>                   | <b>2</b> |
| <b>4</b> | <b>The Config Object</b>             | <b>2</b> |
| 4.1      | Saving . . . . .                     | 2        |
| 4.2      | Time . . . . .                       | 3        |
| 4.3      | Space . . . . .                      | 3        |
| 4.4      | BlackHole . . . . .                  | 3        |
| 4.5      | uldm . . . . .                       | 4        |
| 4.6      | INIT . . . . .                       | 4        |
| 4.7      | BC . . . . .                         | 4        |
| 4.8      | SIM . . . . .                        | 4        |
| 4.9      | LEGACY, IO, RUNTIME . . . . .        | 5        |
| 4.10     | ADVANCED . . . . .                   | 5        |
| 4.11     | SAMPLING . . . . .                   | 5        |
| 4.12     | EXTRAS . . . . .                     | 5        |

|                                           |          |
|-------------------------------------------|----------|
| 4.13 SPECIAL_HALTING_CONDITIONS . . . . . | 6        |
| 4.14 Methods . . . . .                    | 6        |
| <b>5 Units</b>                            | <b>6</b> |
| <b>6 Evolve()</b>                         | <b>6</b> |
| <b>7 Save Flags</b>                       | <b>7</b> |
| <b>8 Output Directory Reference</b>       | <b>8</b> |
| <b>9 N-Body / FDM Coupling</b>            | <b>8</b> |
| <b>10 BH Absorption Sink API</b>          | <b>8</b> |
| <b>11 Known Limitations</b>               | <b>9</b> |
| <b>12 Version History</b>                 | <b>9</b> |

## 1 Overview

PyUltraLight2 (this fork: *BHPaper*) evolves the Schrödinger–Poisson system for Ultralight/Fuzzy Dark Matter (ULDM/FDM),

$$i\hbar\partial_t\psi = \left[-\frac{\hbar^2}{2m}\nabla^2 + m\Phi\right]\psi, \quad (1)$$

$$\nabla^2\Phi = 4\pi Gm|\psi|^2, \quad (2)$$

on a periodic (or isolated) FFT grid via operator splitting (kick–drift–kick), coupled to zero or more classical point-mass “matter particles” (black holes) via a Plummer-softened potential. This fork adds a moving-sink absorption mechanism so a black hole particle can accrete the surrounding ULDM: a local imaginary potential depletes  $\psi$  near the particle, and the lost mass (and, optionally, momentum) is transferred onto the particle’s own dynamical state.

All physical quantities inside the simulation are expressed in *code units* defined by the axion mass `m22` (Section 5). The public entry points are the `Config` class (`PyUltraLight2.Init.Config`) and the `Evolve()` function (`PyUltraLight2.Evolve`).

## 2 Installation and Dependencies

Pure-Python package, no build step. Runtime dependencies: `numpy`, `pyfftw`, `numexpr`, `astropy` (unit constants only). Import as:

```
import PyUltraLight2 as PyUL
```

or, for direct access to the pieces documented here:

```
from PyUltraLight2.Init.Config import Config
from PyUltraLight2.Evolve import Evolve
```

### 3 Quick Start

```
from PyUltraLight2.Init.Config import Config
from PyUltraLight2.Evolve import Evolve

config = Config()
config.Space["Resolution"] = 64
config.Time["TimeDuration"] = 1.0
config.uldm["Solitons"]["Condition"] = [[10, [0, 0, 0], [0, 0, 0], 0]]
config.BlackHole["MatterParticles"]["Condition"] = [
    [0.05, [-0.2, 0, 0], [0.35, 0, 0]]
]
config.BlackHole["Sink"]["Flag"] = True
config.BlackHole["Sink"]["Dynamic"] = True

loc = Evolve(config, Silent=True)
```

This runs a resolution-64 grid with one soliton and one black-hole particle that accretes ULDM via a dynamically-calibrated Bondi–Hoyle sink. Output is written under `loc` (see Section 8).

A config can also be round-tripped to/from a JSON `.uldm` file:

```
config.ToFile("my_run.uldm")
loc = Evolve("my_run.uldm")          # Evolve() also accepts a path string
```

### 4 The Config Object

`Config()` (in `PyUltraLight2.Init.Config`) populates its fields with defaults on construction; every field is a plain nested dict attribute, mutated directly (`config.Space["Resolution"] = 128`). Below, each top-level block is documented as a subsection.

#### 4.1 Saving

**Loc** (str, default `"/"`) Output directory. If left at `"/"`, `Evolve()` auto-generates `./Simulations/{RunSignature}`

**Flags** (str, default `"Minimum"`) Space-separated save flags; see Section 7.

**Format** (str, default `"npz"`) On-disk array format.

**Number** (int, default `-1`) Number of snapshots to save; `-1` means “every integration step”.

#### 4.2 Time

**TimeDuration** (float, default `1`) Simulation duration, in `TimeUnits`.

**StartTime** (float, default `0`) Initial phase-evolution offset for solitons only.

**TimeUnits** (str, default `""`) One of the time units in Table 2 (`""` = code units).

**RKSteps** (int, default `32`) RK4 sub-steps per outer ULDM step for particle integration (`NBodyAdvance`); must be `0`, `1`, or a multiple of `4`.

**StepFactor** (int, default `1`) Coarsens the outer ULDM timestep by this factor.

### 4.3 Space

**Resolution** (int, default 256) Grid points per axis (cubic grid,  $N^3$  total).

**Box.BoxLength** (float, default 1) Box side length, in **Box.LengthUnits**.

**Box.LengthUnits** (str, default "") One of the length units in Table 2.

### 4.4 BlackHole

**MatterParticles.Condition** (list) List of particles, each **[mass, [x,y,z], [vx,vy,vz]]** in **MassUnits / PositionUnits / VelocityUnits**. A particle with **mass == 0** is a passive observer (no gravity).

**MatterParticles.MassUnits / PositionUnits / VelocityUnits** (str) Units for the list above.

**MatterParticles.PlummerRadius** (unused placeholder; see **FieldSmoothing**).

**FieldSmoothing** (float or "Auto", default "Auto") Plummer softening parameter  $a$  (inverse length, code units) used both for particle→field deposition and particle-particle gravity:  $\Phi_{\text{TM}}(r) = -a m / \sqrt{1 + a^2 r^2}$ . "Auto" sets  $a = 2 N / L$  (half a grid cell). A value  $\geq 10^8$  disables softening entirely.

**Sink** Moving absorption sink — see Table 1.

| Field                   | Default      | Meaning                                                                        |
|-------------------------|--------------|--------------------------------------------------------------------------------|
| <b>Flag</b>             | <b>False</b> | Enable the sink.                                                               |
| <b>ParticleIdx</b>      | <b>0</b>     | Index into <b>MatterParticles.Condition</b> the sink follows.                  |
| <b>Amplitude</b>        | <b>0.0</b>   | $V_0$ of the imaginary potential (code units). Ignored if <b>Dynamic</b> .     |
| <b>Radius</b>           | <b>0.05</b>  | Gaussian $\sigma$ of the sink (code length units). Ignored if <b>Dynamic</b> . |
| <b>Feedback</b>         | <b>True</b>  | Add absorbed mass onto the particle's own mass.                                |
| <b>Dynamic</b>          | <b>False</b> | Recompute Amplitude/Radius every step via Bondi–Hoyle (Section 10).            |
| <b>VFloor</b>           | <b>0.05</b>  | Minimum speed used in Dynamic calibration (code units).                        |
| <b>RadiusCap</b>        | <b>None</b>  | Max Radius in Dynamic mode; defaults to $0.25 \times \text{BoxLength}$ .       |
| <b>ConserveMomentum</b> | <b>False</b> | Deposit absorbed momentum onto the particle's velocity.                        |

Table 1: `config.BlackHole["Sink"]` fields.

### 4.5 uldm

**m22** (float, default 1) Axion mass in units of  $10^{-22}$  eV; sets every code unit (Section 5).

**SI** (bool, default **False**) Enable ULDM self-interaction (quartic term).

**LHat** (float, default 0) Self-interaction coupling  $\hat{\lambda}$ , used when **SI** is **True**.

**Solitons.Condition** (list) List of solitons, each **[mass, [x,y,z], [vx,vy,vz], phase]** (phase in units of  $\pi$ ).

**Solitons.Embedded** (list) Embedded soliton profiles (advanced; see `Init/Solitons.py`).

**Solitons.MassUnits / PositionUnits / VelocityUnits** (str) Units for the list above.

**Modifier.UniformFieldAddOn.Flag** (bool, default **False**) Superpose a uniform background density.

**Modifier.UniformFieldAddOn.DensityUnit / DensityValue / UniformVelocity** Background density and bulk velocity, when the flag above is set. Note: a nonzero **UniformVelocity** needs  $vL$  to close periodically (a multiple of  $2\pi$  in code units) or the field develops a boundary discontinuity; prefer giving the *particle* the relative velocity instead of the field when possible.

## 4.6 INIT

**DumpInit / DumpFinal** (bool) Dump the full complex wavefunction to **Init\_psi.npy / Final\_psi.npy** (**DumpInit** also halts the run immediately after initialisation).

**UseInit** (bool) Load a previously-dumped wavefunction from **InitPath** instead of (or blended with) the soliton/particle initial conditions.

**InitPath** (str) Path (without **.npy**) to load from.

**InitWeight** (float, default 1) Blend weight in  $[0, 1]$ , or **-1** to keep only the loaded phase.

## 4.7 BC

**EdgeClear** (bool) Zero  $\psi$  back to background density on the outermost **Resolution**/16 cells every step (crude absorbing boundary).

**IsoP** (bool, default **False**) Use an isolated (zero-padded Green's function) Poisson solve instead of the periodic FFT solve. Also switches particle force interpolation away from periodic wraparound (see Section 9).

**DSponge** (bool) Enable a dispersive sponge layer near the boundary (damps  $\psi$  smoothly in an outer shell).

## 4.8 SIM

**SelfGravity** (bool, default **True**) ULDM feels its own gravity ( $\Phi_{\text{SP}}$ ).

**NBodyInterp** (bool, default **True**) Particles feel the ULDM's gravity. If **False**, particles only feel each other.

**NBodyGravity** (bool, default **True**) ULDM feels the particles' gravity ( $\Phi_{\text{TM}}$  added to the field). *Note:* this does *not* disable direct particle-particle gravity, which always runs inside **NBodyAdvance** regardless of this flag.

## 4.9 LEGACY, IO, RUNTIME

**LEGACY.Shift / NLM** Unused; retained for config-file compatibility with older runs.

**LEGACY.DR** (float, default 1) Soliton profile scale factor passed to **InitSolitonF**.

**IO.Stream / StreamChar** (bool / list) Stream selected **TMState** components to a side file every RK sub-step.

**RUNTIME.GenerateLog** (bool, default **True**) Mirror console output to **evolve.log** in the run directory.

#### 4.10 ADVANCED

**CenterCalc** (bool) Track and log the ULDM density's centre-of-mass every step; also drives 3D resampling around the COM when the corresponding save flags are set.

**ComputeQuad** (bool) Reserved; the quadrupole computation this once triggered was removed as dead code (undefined function).

**ExtPhi** (default 0) Static external potential. Accepts an `ndarray` matching the grid shape, a path to a saved `.npy` array, a callable `f(x,y,z)` (evaluated once) or `f(x,y,z,t)` (time-dependent; evaluated at  $t = 0$  only at present), or `0/None` for none. A nonzero *imaginary* part acts as a static absorbing potential, applied every step as  $\psi \ast e^{-h \text{Im}(\Phi_E)}$  — distinct from, and independent of, the moving **Sink** mechanism.

#### 4.11 SAMPLING

**LengthRatio** (float, default 0.5) Size of the resampled sub-box, as a fraction of the full box, when **ADVANCED.CenterCalc** is set.

**resolR** (int, default 64) Resolution of the resampled sub-box.

#### 4.12 EXTRAS

**PrintEK** (bool) Reserved for kinetic-energy delta printouts.

**MassChange** (bool) Scale particle mass over time via **MassFunc**. *Not compatible with Sink.Feedback:* both write `masslist[i]` each step and will clobber each other.

**MassFunc** (str) Source of a function `f(t) -> multiplier`, used when **MassChange** is set.

**CorrectDrift** (bool) Periodically null out bulk drift velocity inferred from the ULDM centre-of-mass.

**CorrectFreq** (int, default 6) Steps between **CorrectDrift** corrections.

#### 4.13 SPECIAL\_HALTING\_CONDITIONS

**AutoStop** (bool) Halt when a single test particle's velocity crosses zero relative to a uniform background (used for scattering-angle experiments).

**AutoStop2** (bool) Halt when the minimum potential exceeds **WellThreshold** times its initial value (runaway-collapse guard).

**AutoStop3** Reserved, unused.

**KThreshold** (float, default 0.9) Reserved.

**WellThreshold** (float, default 100) See **AutoStop2**.

#### 4.14 Methods

`config.ToFile(path)` Serialise to JSON (`.uldm`).

`config.FromFile(path)` Load from JSON, overwriting all fields.

`config.ULDStepEst(save_number=-1)` Estimate the number of integration steps a run will take, without running it.

`config.RunSignature()` Human-readable run label ("`M{m22}_L{box}{units}_T{dur}{units}@{res}`"), used to auto-name the output directory.

## 5 Units

All code units are derived from the axion mass `m22` (in units of  $10^{-22}$  eV) via `ULDMUniverse` (`Universe/Universe.py`). Passing "" as a unit string means "already in code units". Table 2 lists the recognised physical units per conversion type, used via `convert(value, unit, type)` / `convert_back(value, unit, type)`.

| Type code | Quantity     | Recognised units                                                             |
|-----------|--------------|------------------------------------------------------------------------------|
| l         | length       | m, km, pc, kpc, Mpc, ly, SI                                                  |
| m         | mass         | kg, solar_masses, M_solar_masses*, SI                                        |
| t         | time         | s, yr, kyr, Myr, Gyr, SI                                                     |
| v         | velocity     | m/s, km/s, km/h, c, SI                                                       |
| d         | density      | Crit, MSol/pc <sup>3</sup> , MMSol/kpc <sup>3</sup> , kg/m <sup>3</sup> , SI |
| a         | acceleration | m/s <sup>2</sup> , SI                                                        |
| p         | momentum     | kgm/s, Nr, SI                                                                |

Table 2: Unit strings accepted by `convert/convert_back`. \*M\_solar\_masses =  $10^6 M_\odot$  (mega-solar-masses), *not* one solar mass.

## 6 Evolve()

`Evolve(config_or_path, Silent=False, Message='', Additional_Psi=None, **kwargs)`

`config_or_path` A `Config` instance, or a path string (loaded via `Config().FromFile`).

`Silent` (bool) Suppress interactive prompts (auto-overwrite an existing output folder) and some console output.

`Message` (str) Printed banner message at run start.

`Additional_Psi` (ndarray or None) Added onto the initial  $\psi$  before the main loop, after solitons/particles/uniform background are constructed.

`**kwargs` Config overrides applied before the run. Recognised shorthands: `Saving_Loc`, `DSponge`, `IsoP`, `CenterCalc`, `duration` (map onto the corresponding `Config` fields). Any other keyword either overrides an existing top-level `Config` attribute (if the name matches one) or is set as a new attribute on the `Config` instance.

Returns the output directory path (str). Side effect: writes the full directory tree described in Section 8, including a snapshot of the resolved config to `config.uldm` inside it.

## 7 Save Flags

`Saving.Flags` is a space-separated string of flags from Table 3. The shorthand "Minimum" expands to "Energy 1Density NBody 1GravF".

| Flag       | Output dir/prefix | Description                                                   |
|------------|-------------------|---------------------------------------------------------------|
| 3Density   | R3D               | Full 3D density field $ \psi ^2$                              |
| 3Wfn       | P3D               | Full 3D complex wavefunction $\psi$                           |
| 2Density   | R2D               | Density, xy mid-plane slice                                   |
| Energy     | EGY               | Global energy components (KQ, SI, TM)                         |
| 1Density   | R1D               | Density, y-axis line through centre                           |
| NBody      | NTM               | N-body state (position, velocity)                             |
| 3Grav      | G3D               | Full 3D self-gravity potential $\Phi_{\text{SP}}$             |
| 2Grav      | G2D               | Self-gravity, xy mid-plane slice                              |
| DF         | DYF               | Dynamical friction gradient log                               |
| 2Phase     | A2D               | Phase $\arg(\psi)$ , xy mid-plane slice                       |
| Entropy    | ENT               | Global $-\int \rho \ln \rho dV$                               |
| 1Grav      | G1D               | Self-gravity, y-axis line                                     |
| 3GravF     | F3D               | Full 3D total potential $\Phi_{\text{SP}} + \Phi_{\text{TM}}$ |
| 2GravF     | F2D               | Total potential, xy mid-plane slice                           |
| 1GravF     | F1D               | Total potential, y-axis line                                  |
| 3DensityRS | R3R               | Subsampled 3D density around COM                              |
| 3WfnRS     | P3R               | Subsampled 3D wavefunction around COM                         |
| 2EnergyTot | E2T               | Energy density, xy mid-plane slice                            |
| 2EnergyKQ  | E2K               | Kinetic-quantum energy density slice                          |
| 2EnergySI  | E2G               | Self-interaction energy density slice                         |
| ULDCOM     | UCM               | ULDM centre-of-mass position                                  |
| 2DensityRS | R2R               | Subsampled density, 2D slice                                  |

Table 3: Recognised `Saving.Flags` tokens.

## 8 Output Directory Reference

Given output directory `loc` (returned by `Evolve()`):

`loc/config.uldm` The resolved `Config`, as JSON.

`loc/evolve.log` Console log (if `RUNTIME.GenerateLog`).

`loc/Outputs/{DIRNAME}/` One subdirectory per requested save flag (Table 3), containing numbered `{DIRNAME}_#NNN.npy` snapshots.

`loc/Outputs/ULDMass.npy` Total ULDM mass on the grid ( $V_{\text{cell}} \sum \rho$ ) at every save point. Always written.

`loc/Outputs/BHMass.npy` The sink particle's mass at every save point, only when `Sink.Flag` is set.

`loc/Outputs/egylist.npy`, `egpcmlist.npy`, `egpcmMlist.npy`, `egpsilist.npy`, `ekandqlist.npy`, `masseslist.npy`  
Energy bookkeeping time series, only when `Energy` is in `Saving.Flags`.

loc/Outputs/Entro.npy Entropy time series, only when `Entropy` is in `Saving.Flags`.  
loc/Init\_psi.npy, loc/Final\_psi.npy Full complex wavefunction dumps, only when `INIT.DumpInit`  
/ `INIT.DumpFinal` are set.  
loc/StoppingTime.uldm Written when `AutoStop` triggers (or -1 if it never did).

## 9 N-Body / FDM Coupling

Particle motion is integrated by `PyUltraLight2.Integration.NBodyAdvance`:

```
NBodyAdvance(TMState, h, masslist, phiSP, smoothing, lengthC, resol, NS,
             loc='', Stream=False, StreamChar='', Periodic=True)
```

which RK4-subcycles (`Time.RKSteps` sub-steps) the ODE

$$\ddot{\mathbf{r}}_i = -\nabla\Phi_{\text{SP}}(\mathbf{r}_i) - \sum_{j \neq i} m_j \nabla \left[ -\frac{a m_j}{\sqrt{1 + a^2 r_{ij}^2}} \right], \quad (3)$$

using  $\Phi_{\text{SP}}$  frozen at the value from the start of the current outer ULDM step. The gradient of  $\Phi_{\text{SP}}$  is sampled by 2nd-order finite difference on the grid plus trilinear interpolation. `Periodic` (= `not config.BC.IsoP`, set automatically by `Evolve()`) selects whether the local force-sampling stencil wraps periodically across the box boundary (the periodic Poisson-solve case) or, for an isolated potential, falls back to a zero gradient within two grid cells of any face.

## 10 BH Absorption Sink API

Implemented in `PyUltraLight2.Particles.Accretion`, called once per step from `Evolve()` (after the particle's position is advanced for that step) when `Sink.Flag` is set.

```
ApplySink(psi, rho, TMState, masslist, SinkIdx, SinkAmplitude, SinkRadius,
          xarray, yarray, zarray, h, Vcell, Feedback=True,
          Dynamic=False, VFloor=0.05, RadiusCap=None,
          ConserveMomentum=False, kxarray=None, kyarray=None, kzarray=None)
-> (psi, rho, masslist)
```

Centred on the sink particle's current position  $\mathbf{r}_0$ , a Gaussian imaginary potential

$$V_{\text{sink}}(\mathbf{r}) = A \exp\left(-\frac{|\mathbf{r} - \mathbf{r}_0|^2}{2R^2}\right) \quad (4)$$

is applied as  $\psi \leftarrow \exp(-h V_{\text{sink}}) \psi$ . When `Feedback`, the mass removed from the grid,  $\Delta M = V_{\text{cell}} \sum (\rho_{\text{before}} - \rho_{\text{after}})$ , is added to the particle's mass. When `ConserveMomentum` (needs the spectral wavenumber grids `kxarray/kyarray/kzarray`), the absorbed ULDM current  $\mathbf{j} = \text{Im}(\psi^* \nabla \psi)$  is integrated against the same absorbed fraction and deposited onto the particle's velocity as a mass-weighted (rocket-equation) update, so the sink also imparts drag/recoil rather than being a pure mass leak.

```
BondiHoyleCalibration(M, v, cs=0.0) -> (Amplitude, Radius)
```

Returns  $(A, R)$  such that `ApplySink`’s early-time absorption rate matches the classical Bondi–Hoyle–Lyttleton rate

$$\dot{M}_{\text{BHL}} = \frac{4\pi(GM)^2\rho}{(v^2 + c_s^2)^{3/2}}, \quad G = 1 \text{ in code units}, \quad (5)$$

via

$$R = \frac{2M}{v^2 + c_s^2}, \quad A = \frac{(v^2 + c_s^2)^{3/2}}{8\sqrt{2\pi} M}. \quad (6)$$

When `Sink.Dynamic` is set, `ApplySink` calls this every step with the particle’s *current* mass and speed (floored at `VFloor`, and  $R$  optionally capped at `RadiusCap`) instead of using a single fixed  $(A, R)$  for the whole run. This is a dimensional match, not a first-principles derivation — verify empirically for any new regime (mass ratio, resolution, duration) before trusting it.

## 11 Known Limitations

See `CHANGES.md` in the repository root for the full, dated history. Notable open items at the time of writing:

- `EXTRAS.MassChange` and `Sink.Feedback` both write `masslist[i]` each step and will clobber each other if both enabled.
- `Sink.Amplitude/Sink.Radius` are plain code-unit numbers with no unit-string conversion (unlike `FieldSmoothing`).
- Force-sampling accuracy for the particle is grid-resolution limited (2nd-order finite difference + trilinear), while the particle’s own contribution to the field ( $\Phi_{\text{TM}}$ ) is exact analytically; this asymmetry is most relevant when `Sink.Dynamic` shrinks the Bondi radius toward the grid scale.
- `SIM.NBodyGravity=False` only disables the particle→field channel; direct particle-particle gravity inside `NBodyAdvance` is unconditional.

## 12 Version History

See `CHANGES.md` in the repository root.